

Module 7

Version Comparison and Migration

Learning objectives

- Compare Verilog and SystemVerilog across IEEE versions side-by-side, plan migration between standar...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Side-by-Side Comparison: Sam"]

T1 --> N["Next module in course"]

Compare Verilog and SystemVerilog across IEEE versions side-by-side, plan migration between standards, and choose the right version for you...

Overview

- This module ties together Modules 1–6 by comparing the same design across standards (1364-1995 thro...

How to learn this module

Suggested learning path (1/2)

- Skim this document — Goal, Overview, and Topics
Covered set the IEEE scope for Module 7.
- Study design architecture — See how DUTs, examples, and testbenches fit together under ``module7/``.
- Work through labs in order — Open ``module7/EXAMPLES.md`` and run each ``make clean && make run`` from...
- Run the full module script — ``./scripts/module7.sh`` from the repo root to simulate all examples and...
- Complete the exercises — Try each exercise in this document before reading solutions or peeking at...

Suggested learning path (2/2)

- Compare side-by-side — Open 1364 and 1800 versions of the same design and note port style, types, a...
- Review Common Pitfalls — Can you explain each mistake and the fix?
- Self-check — Use ``module7/CHECKLIST.md`` before starting the next module.

Syllabus topics

1. Side-by-Side Comparison: Same Design Across Versions

- See docs/MODULE7.md

Hands-on examples

Module 7 toolchain check

```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog  
-V 2>&1 | head -1)"
```

Terminal: module7_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Example 7: Migration 1364→1800

- Before/after: wire/reg, always @* vs logic, always_comb, unique case
- Order of steps, testing, regression

Demo: Migration 1364→1800

```
$ cd module7/examples/migration && make clean && make run
```

Terminal: ex07_migration

```
rm -f top
iverilog -g2012 -o top migration_steps.sv
vvp top
Module 7: Migration 1364 -> 1800 (steps: wire/reg->logic, @*->always_comb, unique case)
  sel=10 y_before=04 y_after=04
  PASS: same behavior
migration_steps.sv:53: $finish called at 10 (1s)
migration_steps.sv:30: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 8: Migration 1995→2001

- Before: non-ANSI, @(a or b or sel). After: ANSI,
always @*
- ANSI ports, implicit sensitivity

Demo: Migration 1995→2001

```
$ cd module7/examples/migration_1995_to_2001 && make clean && make run
```

```
Terminal: ex08_migration_1995_to_2001  
rm -f top  
iverilog -o top migration_1995_2001.v  
vvp top  
Module 7: Migration 1995 -> 2001 (ANSI ports, always @*)  
  sel=0: y1995=0 y2001=0  
  sel=1: y1995=1 y2001=1  
  PASS: same behavior  
migration_1995_2001.v:40: $finish called at 20 (1s)
```


Example 9: Case Versions

- Same 4:1 mux: 1364 case+default vs 1800 unique case
- case semantics; unique case for tool checking

Demo: Case Versions

```
$ cd module7/examples/case_versions && make clean && make run
```

Terminal: ex09_case_versions

```
rm -f top
iverilog -g2012 -o top case_versions.sv
vvp top
Module 7: case 1364 (case+default) vs 1800 (unique case)
  sel=01 y1364=02 y1800=02
  sel=11 y1364=08 y1800=08
  PASS: same behavior
case_versions.sv:54: $finish called at 20 (1s)
case_versions.sv:30: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 10: Version Table

- Single reference: construct vs standard (1995–2017)
- When each feature was introduced; minimum revision

Demo: Version Table

```
$ cd module7/examples/version_table && make clean && make run
```

Terminal: ex10_version_table

```
rm -f top
iverilog -g2012 -o top version_table.sv
vvp top
Module 7: Version table (construct vs standard)
Port style: 1995 non-ANSI | 2001+ ANSI
Type:      1364 wire/reg | 1800 logic (single driver)
Combinational: 1995 @(a or b) | 2001 @* | 1800 always_comb
Sequential: 1364 always @(posedge clk) | 1800 always_ff
Connectivity: 1364 many ports | 1800 interface+modport
Case:      1364 case | 1800 unique/priority case
Reference: 1800-2017 single LRM (Verilog + SystemVerilog)
version_table.sv:16: $finish called at 0 (1s)
```

Example 11: Version Selection

- Runnable reminder of version-selection checklist
(constraints, decision, document)
- See MODULE7.md for full checklist
- Same D flip-flop in 1995, 2001, 2005, 1800
(sequential block style)
- always @(posedge clk) vs always_ff; wire/reg vs logic
- Same tiny FSM in 1364 vs 1800 (case vs unique case, always_ff)
- Sequential FSM; 1800 always_ff and unique case

Demo: Version Selection

```
$ cd module7/examples/version_selection && make clean && make run
```

Terminal: ex11_version_selection

```
rm -f top
iverilog -g2012 -o top version_selection.sv
vvp top
Module 7: Version selection checklist (reminder)
  Constraints: tools (sim/synth), IP/legacy, team, target (FPGA/ASIC)
  Verilog-only (1364): wire/reg, always @*, no logic/interfaces/packages
  SystemVerilog design (1800): logic, always_comb/always_ff, packages, unique/priority case
  Decision: state one revision (e.g. IEEE 1800-2017); define subset if needed
  Document: style guide or README; reference LRM and tool manuals
  See MODULE7.md for full version-selection checklist.
version_selection.sv:15: $finish called at 0 (1s)
```

Example 14: Migration Checklist

- Runnable reminder of 1364→1800 migration steps (pre, per-module, post)
- See MODULE7.md for full migration checklist

Demo: Migration Checklist

```
$ cd module7/examples/migration_checklist && make clean && make run
```

Terminal: ex14_migration_checklist

```
rm -f top
iverilog -g2012 -o top migration_checklist.sv
vvp top
Module 7: Migration checklist (1364 -> 1800 design subset)
  Pre: target standard, baseline sim/synth, list files/hierarchy
  Per-module: ports wire/reg->logic, @*->always_comb, posedge clk->always_ff
  Per-module: case->unique/priority case, optional interface/package, no defparam
  Post: full regression, update style guide, document 1364-only blocks
  See MODULE7.md for full migration checklist.
migration_checklist.sv:14: $finish called at 0 (1s)
```


Example 15: Incremental Migration

- Two steps: step1 1364 style, step2 1800 style; same mux, compare outputs
- Migrate one construct at a time; test after each step

Demo: Incremental Migration

```
$ cd module7/examples/incremental_migration && make clean && make run
```

```
Terminal: ex15_incremental_migration  
rm -f top  
iverilog -g2012 -o top incremental_migration.sv  
vvp top  
Module 7: Incremental migration (step1=1364, step2=1800)  
  sel=11 y_step1=08 y_step2=08  
  PASS: same behavior after migration  
incremental_migration.sv:54: $finish called at 10 (1s)  
incremental_migration.sv:31: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 16: Port Style Compare

- Same 2:1 mux: non-ANSI (1995) vs ANSI (2001) ports only
- Port declaration style; no other change

Demo: Port Style Compare

```
$ cd module7/examples/port_style_compare && make clean && make run
```

```
Terminal: ex16_port_style_compare  
  
rm -f top  
iverilog -o top port_style_compare.v  
vvp top  
Module 7: Port style - non-ANSI vs ANSI (same mux)  
  sel=0: y_non_ansi=0 y_ansi=0  
  sel=1: y_non_ansi=1 y_ansi=1  
  PASS: same behavior  
port_style_compare.v:39: $finish called at 20 (1s)
```

Example 17: No defparam

- 1364-2005: parameter override at instantiation
 `#(.WIDTH(n));` avoid defparam
- defparam deprecated; use instantiation override

Demo: No defparam

```
$ cd module7/examples/no_defparam && make clean && make run
```

Terminal: ex17_no_defparam

```
rm -f top
iverilog -o top no_defparam.v
vvp top
Module 7: No defparam - parameter override at instantiation (1364-2005)
  count8= 5 count16= 5
  Use #(.WIDTH(n)) at instantiation; avoid defparam.
no_defparam.v:36: $finish called at 70 (1s)
```

Common pitfalls

Migrating Everything at Once

- Mistake: Converting a large codebase from 1364 to 1800 in one big change.
- Reality: Hard to debug; regressions are difficult to isolate.
- Correct: Migrate incrementally (one module or one feature at a time); run regression after each step...
- Why: Reduces risk and keeps a clear baseline.

Ignoring Tool Support

- Mistake: Choosing 1800-2017 and using constructs the synthesis or simulator does not support.
- Reality: Compile or elaboration errors; or tool-specific behavior.
- Correct: Check tool manuals for supported revision and synthesizable subset; align RTL with that.
- Why: Ensures the project builds and matches tool expectations.

Breaking Interfaces During Migration

- Mistake: Changing port list or interface of a block while migrating its internals, so that other mo...
- Reality: Large ripple of changes; integration failures.
- Correct: Keep external interface (ports or interface modport) unchanged during internal migration;...
- Why: Limits scope of change and preserves compatibility with rest of design.

Mixing Revisions in One File

- Mistake: Using 1995 ports in one module and 1800 logic in another in the same file without a clear...
- Reality: Confusing for readers and tools; possible tool-dependent behavior.
- Correct: Use one project standard per file (or per project); document the standard in the style gui...
- Why: Consistency and predictable tool behavior.

Forgetting to Test After Migration

- Mistake: Changing syntax (e.g. wire→logic, always @*→always_comb) and not re-running simulation or...
- Reality: Subtle bugs (e.g. latch, wrong sensitivity) can appear.
- Correct: Run full regression (simulation and synthesis) after every migration step; compare with ba...
- Why: Catches functional or synthesis differences early.

Practice & assessment

What you should know

- ✓ Compare the same design across 1364-1995, 1364-2001, 1364-2005, and 1800 (design subset) and list...
- ✓ Compare connectivity style: many ports (1364) vs interface (1800)
- ✓ Plan and execute migration from 1364 to 1800 design subset (or between 1364 revisions) using the...
- ✓ Choose a project standard (revision and subset) based on tool support and constraints
- ✓ Use the version-selection checklist for new or existing projects
- ✓ Describe how to extend the course when a new IEEE 1800 revision is released

Exercises

- Side-by-Side Mux
- Side-by-Side Connectivity
- Migration One Block
- Version Selection
- Version Table

Exercises

- Migrating Everything at Once
- Ignoring Tool Support
- Breaking Interfaces During Migration
- Mixing Revisions in One File
- Forgetting to Test After Migration

Summary & next steps

- Compare Verilog and SystemVerilog across IEEE versions side-by-side, plan migration between standar...
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: Next module in course